

Replicating: HPCC: High Precision Congestion Control

Team Members:

Andrea Bellani (andrea1.bellani@mail.polimi.it);

Andrea Migliorini (andrea1.migliorini@mail.polimi.it).

Source Paper:

Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, M. Yu: HPCC: High Precision Congestion Control. In SIGCOMM '19, August 19–23, 2019, Beijing, China, © 2019 Association for Computing Machinery.

Project:

[link to our repository](#)

1. Introduction

HPCC: High Precision Congestion Control presents a novel approach to Congestion Control for Datacenter networks. In particular, it is a window-based Congestion Control algorithm that leverages INT-MD (*In-band Network Telemetry - EMbed Data*) to get precise queue lengths information.

In the paper, authors explain the motivation behind HPCC, its design and test its effectiveness with both testbed and NS-3 simulations, comparing it with state-of-art Congestion Control algorithms for Datacenter Networks.

1.1. Challenges in achieving High Precision Congestion Control for RDMA applications

The demand of high-speed Datacenter networks is driven by two main trends, according to both state-of-art and authors experience:

- new *resource disaggregation* and *heterogeneous computing* architectures require both low network latency levels and high network bandwidth;
- new applications (eg. large-scale ML training on high computation speed devices) that periodically transfer large volume data with super-fast storage and computation speeds, often making the network the real performance bottleneck.

To sustain these requirements, new approaches started to offload network stacks into hardware, authors in particular started to deploy large-scale networks with *RDMA* (Remote Direct Memory Access) over *RoCEv2* (RDMA over Converged Ethernet Version 2). HPCC tries to cope with the fundamental challenges authors have experienced with RoCEv2:

- many flows starting at line rate causing severe congestions and deep packet queueing;
- preventing PFC pauses, since they can cause huge traffic drops;
- short flows experiencing really long latency due to deep packet queueing;
- limitations of other state-of-art Congestion Control mechanisms (mainly DCQCN and TIMELY):
 - slow convergence of iterative methods that leverage coarse-grained feedback signals (eg. ECN);
 - techniques inherently based on mechanisms that wait queues build-ups (eg. ECN marking or detecting RTT increasing) to adjust sender rates;
 - complicated parameter tuning due to an high number of parameters or lacking of simple rules of thumb to set them.

HPCC is a Congestion Control algorithm for RDMA designed to solve these challenges.

1.2. How HPCC works

The main keypoint of HPCC is the usage of INT-MD (*In-band Network Telemetry - Embed Data*) to obtain precise link loads in particular, each switch in the path enqueues an INT header that contains various information which sender can use to proactively adjust its sending rate (eg. Queue Length). Instead of relying on heuristics or iterative approaches, senders receive a per-ACK updates to rapidly adjust the sending window, with a minimal set of algorithm's parameters.

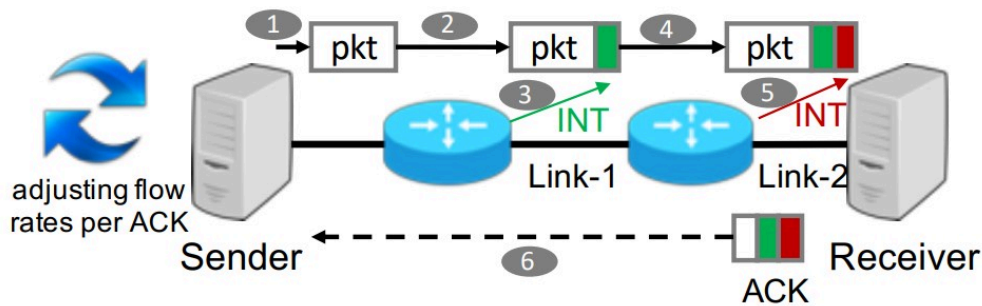


Figure 4: The overview of HPCC framework.

More in detail, an HPCC sender adjusts its sending window if and only if a new ACK arrives. We can distinguish two different phases in the HPCC algorithm (names are invented by us to easily distinguish them):

- *agnostic phase* : the sender keeps increasing its window regardless of network utilization U ;
- *adaptive phase* : the sender adapts its window adjusting it of a factor $\frac{\eta}{U}$, where U is calculated using information from the INT headers collected over the flow path (note that if $U \ll \eta$ the window increases, which is fine since the network is not congested).

Senders normally start in the agnostic phase and pass to the adaptive one after a certain number of RTTs or if $U \geq \eta$.

An important design choice of HPCC lies in when the sender reacts. In particular, the sender always adjusts its window from what we can call a *reference window* W^C that is updated nearly every-RTT (to be more precise, it is updated only when the ACK of the first packet sent from the last W^C update is received). This technique is implemented to achieve what authors call *fast reaction without over-reaction*. In other words, it is essential that the sender quickly reacts to changes, but adjusting its window completely at every ACK received is ... because ... (mettere la cosa che tutte le code sono diverse ogni RTT). So, the window adjustment is performed at per-ACK starting from a reference window (W^C) that is updated nearly every-RTT.

Note that HPCC requires just 3 parameters to be configured, which meaning is quite easy to understand:

Parameter	Meaning	How to set it
η	Network utilization threshold that triggers the *adaptive phase*	Nearly to 0.95, depening on how critical the
W_{AI}	Additive Increase window to ensure fairness	Rule of thumb: $W_{AI} = \frac{W_{init} \times (1-\eta)}{N}$, where N is the (maximum) number of expected concurrent flows on a single link
$maxStage$	Number of maximum stage before passing to the adaptive phase	Generally low

1.3. HPCC main contributions

The main contribution of the paper is the leverage of INT to improve the Congestion Control on large-scale and high-speed networks. The paper present a detailed description of the in-production Congestion Control mechanism running in the AliBaba Group datacenters, which can provide insights for the design of new mechanisms that leverage INT (which was not leveraged by other state-of-art CC mechanisms).

Moreover, the paper also share AliBaba production experiences on the difficulties to operate RDMA networks with the state-of-the-art high-speed CC and

2. Selected Result

2.1. Fairness and queue size with W_{AI}

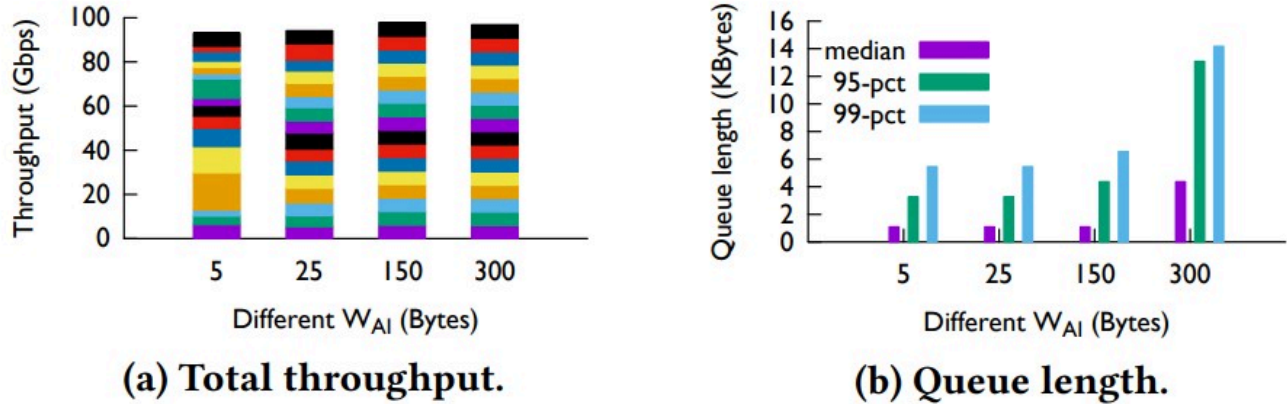


Figure 14: Fairness and queue size with W_{AI} .

Considering an incast of 16 flows, Figure 14a shows the average throughput for different W_{AI} values, Figure 14b shows length's percentiles of the incast Queue

2.2. Different ways of reacting to ACK.

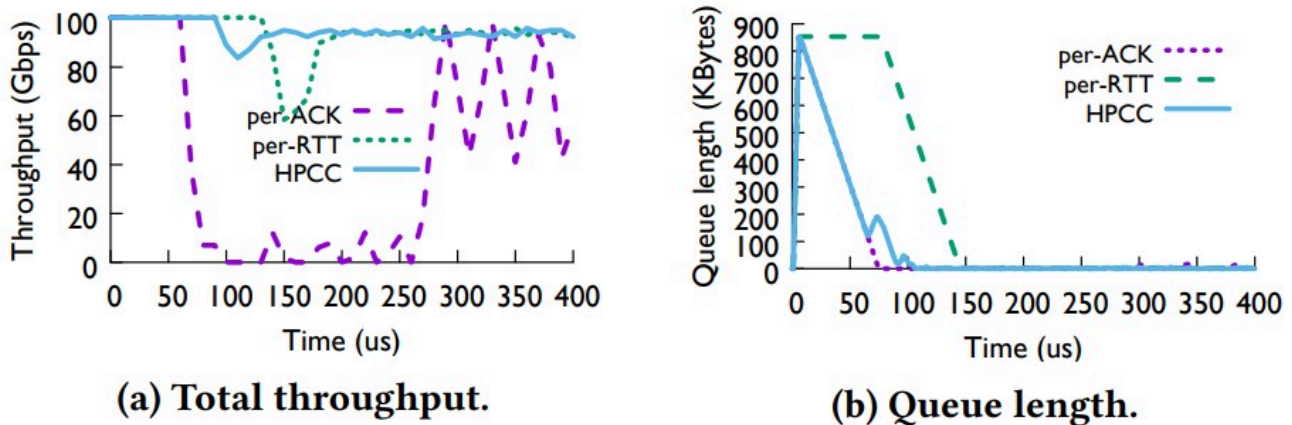


Figure 13: Different ways of reacting to ACK.

Considering an incast of 16 flows, Figure 13a and Figure 13b shows respectively the aggregated throughput and Queue Length running HPCC with different reacting policies

3. Environment Setup

Note: This section should contain enough information to allow someone else to reproduce *your* report. Share hardware and/or software setup relevant to your experiment. For example:

Hardware Environment:

CPU, Memory, Storage, Network, Cloud / local / cluster, Any relevant micro-architectural details

Software Environment

OS version, Kernel version, Compiler version, Libraries, Dependencies, Paper artifact used (Yes/No; version/commit hash)

Configuration Parameters:

- Workload configuration
- Dataset
- Runtime parameters and flags

Deviations from the Original Setup:

Clearly describe any difference between papers and your experiment environment.

- Hardware differences
- Software version differences
- Dataset substitutions
- Unavailable components

Explain why these deviations were necessary.

If something was **missing in the original paper**, state it. For example:

The paper does not specify X. We assumed Y (or explored range a to b).

4. Experiment Result

Explain how your experiment was conducted and then what results you acquired. Afterwards, compare your results with those of the paper and state your takeaways.

Step-by-step description:

1. Execution procedure
2. Measurement method
3. Number of runs
4. Statistical treatment (mean, median, CI, etc.)

Also Describe:

- How did you ensure correctness (did you check also other metrics to make sure the experiment is running correctly?)
- Did you do any debugging? Discuss issues you faced and how you overcame them (if applicable consider allocating a subsection for this item)

Share your result and compare them with the paper's. Then discuss your takeaways.

For comparison include:

- Graph(s) or table(s)
- Matching axes and units with the source paper
- Error bars if applicable
- You may want to report difference with the original results (e.g., absolute number or percentage).

For example:

 The figure shows that method A improves throughput compared to method B
Figure 2: The figure shows that method A improves throughput compared to method B

 Our reproduction of Figure 1 shows results with the similar trend as claimed by the paper
Figure 3: Our reproduction of Figure 1 shows results with the similar trend as claimed by the paper

Reminder: the goal is not achieve the exact results of the paper, but to do a rigorous experiment with similar assumptions from the source paper and gain insight. The insight can be correctness of work, failure to reproduce same results, or even infeasibility of doing such experiment for interesting reasons.

5. Further Exploration

In this project you are required to also explore a research question of your own. Either:

1. Take the same test with different input workload or a variation of a test that is not present in the paper and comment the results you obtain
2. Implement a new feature on top of the system you evaluated and show a figure showing the performance

Discuss which approach you take, and what you explored. Explain what was your motivation and importance of your question.

5.1. Methodology and Result

Report the experiment you designed for answering the question and share the result you got.

Include:

- Graph(s) or table(s)
- How the experiment was conducted (share the details)
- What did you discover?

6. Reproducibility Assessment of the Paper

Evaluate the paper itself:

- Was the methodology clearly described?
- Was the artifact usable?
- How difficult was reproduction?

6.1 Paper evaluation

6.2 Replication Package evaluation

7. Conclusion

Conclude the report by mentioning the takeaways of experiments you did